UNIT IV

STORAGE ALLOCATION

LECTURE NO: 1

STORAGE ALLOCATION STRATEGIES

DEFINITION OF STORAGE ALLOCATION

Definitions of storage allocation. (computer science) **the assignment of particular areas of a magnetic disk to particular data or instructions**. synonyms: allocation. type of: assigning, assignment. the act of distributing something to designated places or persons.

Difference between Static Allocation and Heap Allocation

Static Allocation:

Static allocation is an allocation procedure which is used for allocation of all the data objects at compile time. In this type of allocation, allocation of data objects is done at compile time only.

As in static allocation compiler decides the extent of storage which cannot be change with time, hence it is easy for the compiler to know the addresses of these data objects in the activation records at later stage. Static allocation is implemented in FORTRAN.

Heap Allocation:

Heap allocation is an allocation procedure in which heap is used to manage the allocation of memory. Heap helps in managing the dynamic memory allocation. In heap allocation, creation of dynamic data objects and data structures is also possible as same as stack allocation. Heap allocation overcomes the limitation of stack allocation.

It is possible to retain the value of variables even after the activation record in heap allocation strategy which is not possible in stack allocation. It maintains a linked list for the free blocks and reuse the deallocated space using best fit.

The difference between Static Allocation and Heap Allocation are as follows:

S.No.	Static Allocation	Heap Allocation
1	Static allocation allocates memory on the basis of size of data objects	Heap allocation make use of heap for managing the allocation of memory at run time.
2	In static allocation, there is no possibility of creation of dynamic data structures and objects.	In heap allocation, dynamic data structures and objects are created.
3	In static allocation, the names of the data objects are fixed with storage for addressing.	Heap allocation allocates contiguous block of memory to data objects.
4	Static allocation is simple, but not efficient memory management technique.	Heap allocation does memory management in efficient way.
5	Static allocation strategy is faster in accessing data as compared to heap allocation.	While heap allocation is slow in accessing as there is chance of creation of holes in reusing the free space.
6	Static allocation is inexpensive, it is easy to implement.	While heap allocation is comparatively expensive.

ACTIVATION RECORDS

Runtime Environments

A translation needs to relate the static source text of a program to the dynamic actions that must occur at runtime to implement the program. The program consists of names for procedures, identifiers etc., that require mapping with the actual memory location at runtime.

Runtime environment is a state of the target machine, which may include software libraries, environment variables, etc., to provide services to the processes running in the system.

Activation Tree

A program consists of procedures, a procedure definition is a declaration that, in its simplest form, associates an identifier (procedure name) with a statement (body of the procedure). Each execution of procedure is referred to as an activation of the procedure.

Lifetime of activation is the sequence of steps present in the execution of the procedure. If a‘ and b‘ be two procedures, then their activations will be non-overlapping (when one is called after other) or nested (nested procedures).

A procedure is recursive if a new activation begins before an earlier activation of the same procedure has ended. An activation tree shows the way control enters and leaves activations.
Properties of activation trees are :-

- Each node represents an activation of a procedure.
- The root shows the activation of the main function.
- The node for procedure ‗x‘ is the parent of node for procedure ‗y‘ if and only if the control flows from procedure x to procedure y.

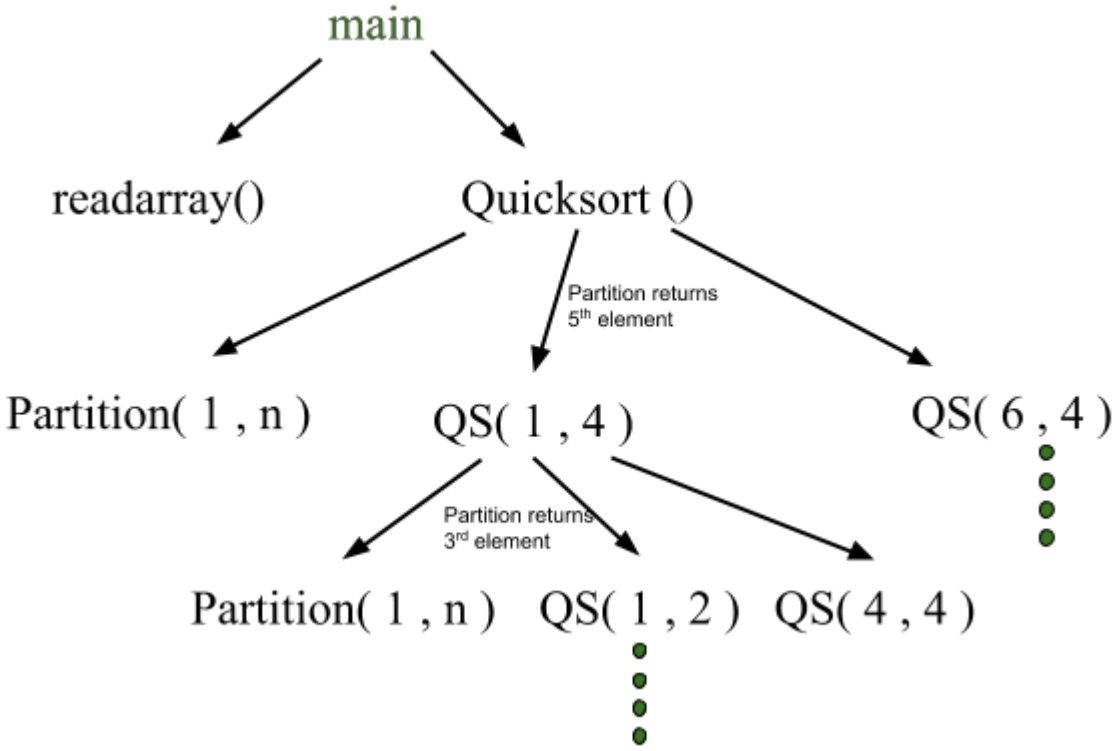
Example – Consider the following program of Quicksort

```
main() {
    Int n;
    readarray();
    quicksort(1,n);
}

quicksort(int m, int n) {

   Int i= partition(m,n);
   quicksort(m,i-1);
   quicksort(i+1,n);
}
```

The activation tree for this program will be:



First main function as root then main calls read array and quicksort. Quicksort in turn calls partition and quicksort again. The flow of control in a program corresponds to the depth first traversal of activation tree which starts at the root.

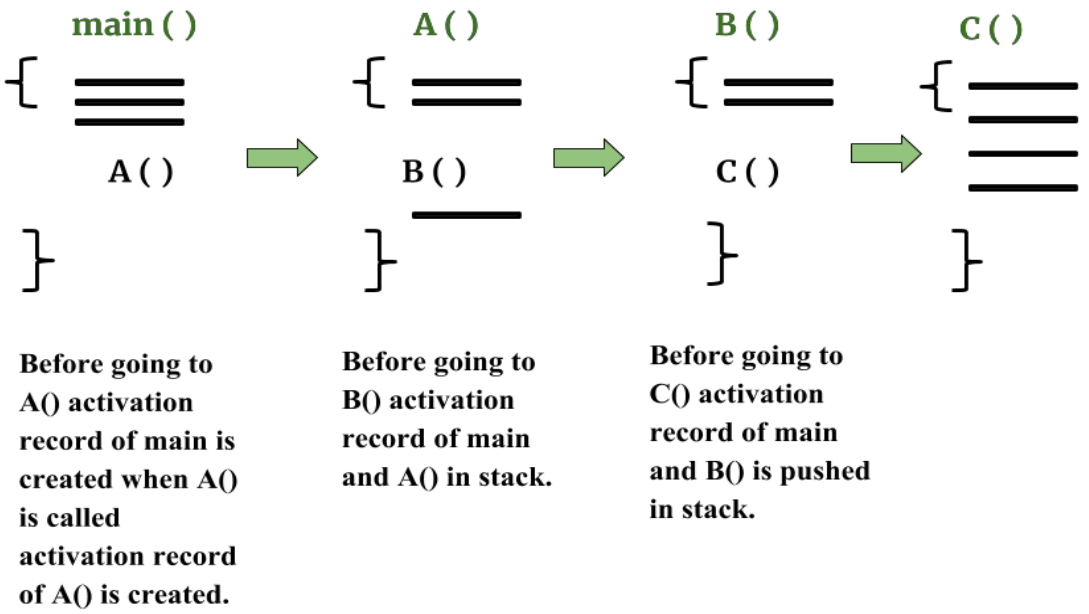
LECTURE NO: 2

ACCESSING LOCAL NAMES IN A BLOCK STRUCTURE LANGUAGE

CONTROL STACK AND ACTIVATION RECORDS

Control stack or runtime stack is used to keep track of the live procedure activations i.e. the procedures whose execution have not been completed. A procedure name is pushed on to the stack when it is called (activation begins) and it is popped when it returns (activation ends).

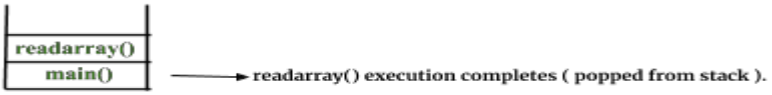
Information needed by a single execution of a procedure is managed using an activation record or frame. When a procedure is called, an activation record is pushed into the stack and as soon as the control returns to the caller function the activation record is popped.



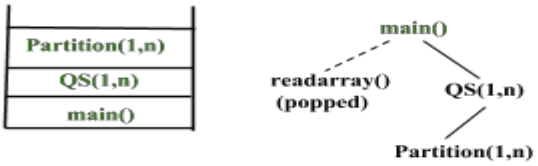
A general activation record consists of the following things:

- **Local variables:** hold the data that is local to the execution of the procedure.
- **Temporary values**: stores the values that arise in the evaluation of an expression.
- **Machine status:** holds the information about status of machine just before the function call.
- **Access link (optional):** refers to non-local data held in other activation records.
- **Control link (optional):** points to activation record of caller.
- **Return value:** used by the called procedure to return a value to calling procedure
- Actual parameters

Control stack for the above quicksort example:

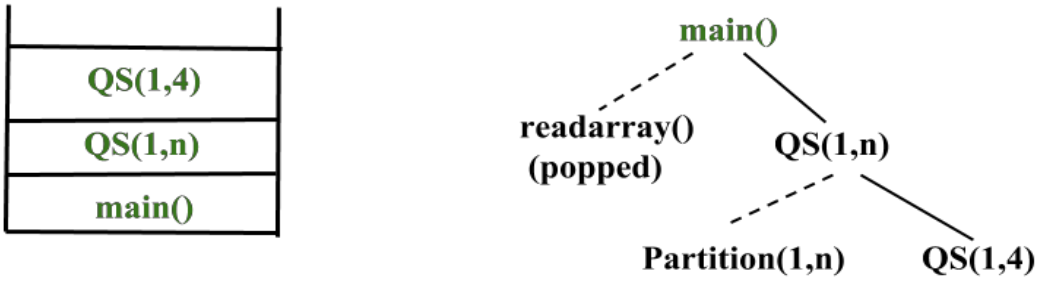


QS is called so it Enters the Stack.



Partition Execution completed (popped out of stack)

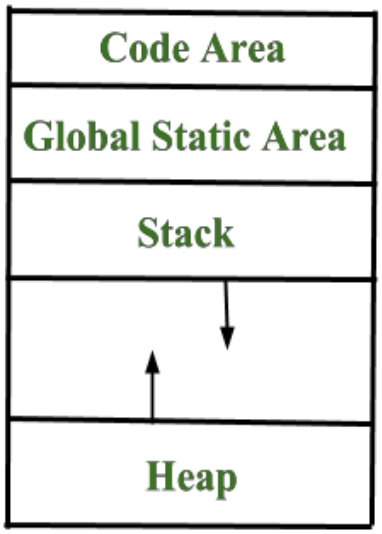
Now QS is called again so it enters the Stack.



SUBDIVISION OF RUNTIME MEMORY

Runtime storage can be subdividing to hold:

- Target code- the program code, it is static as its size can be determined at compile time
- Static data objects
- Dynamic data objects- heap
- Automatic data objects- stack



The Stack grows towards Higher Memory.

Heap grows towards lower Memory.

STORAGE ALLOCATION TECHNIQUES

I. Static Storage Allocation

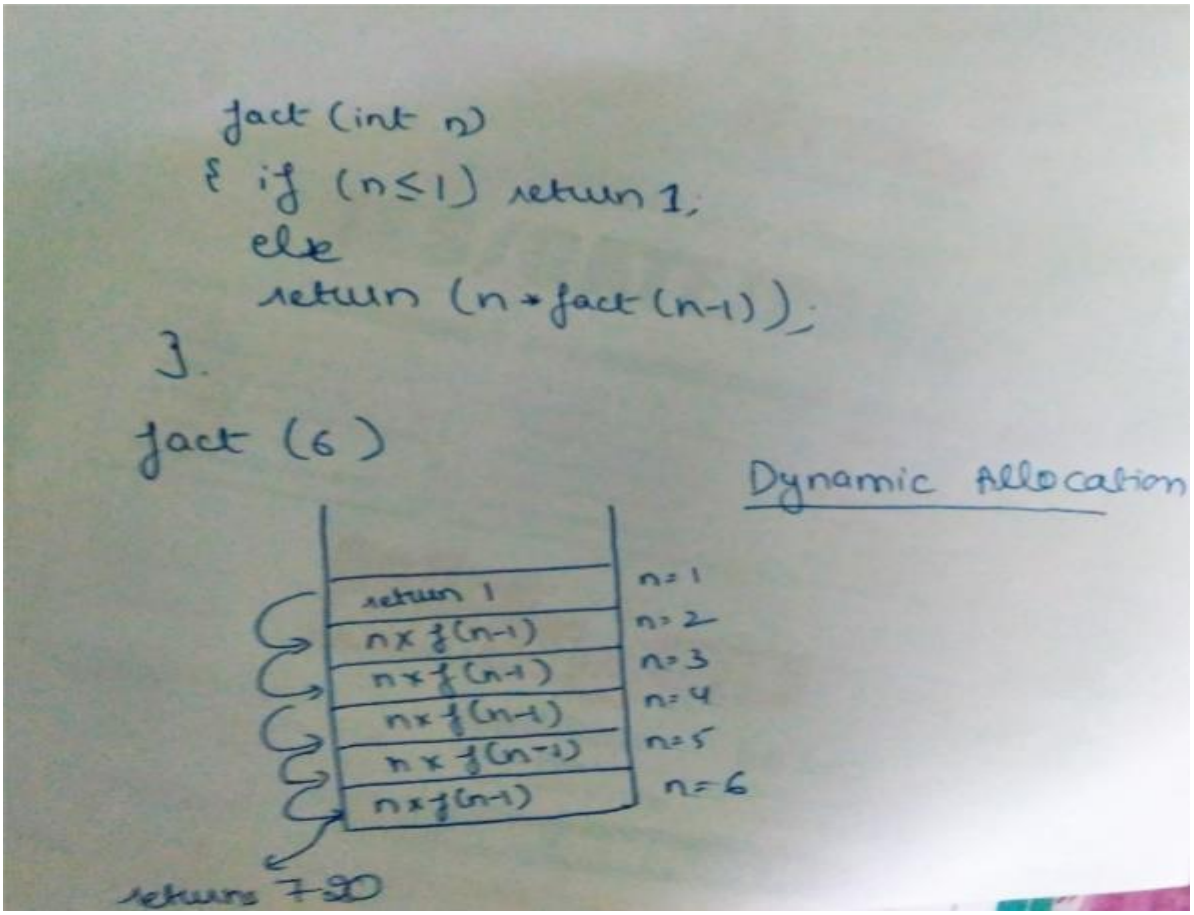
- For any program if we create memory at compile time, memory will be created in the static area.
- For any program if we create memory at compile time only, memory is created only once.
- It doesn‘t support dynamic data structure i.e memory is created at compile time and deallocated after program completion.
- The drawback with static storage allocation is recursion is not supported.
- Another drawback is size of data should be known at compile time
  Eg- FORTRAN was designed to permit static storage allocation.

II. Stack Storage Allocation

- Storage is organised as a stack and activation records are pushed and popped as activation begin and end respectively. Locals are contained in activation records so they are bound to fresh storage in each activation.
- Recursion is supported in stack allocation

III. Heap Storage Allocation

- Memory allocation and deallocation can be done at any time and at any place depending on the requirement of the user.
- Heap allocation is used to dynamically allocate memory to the variables and claim it back when the variables are no more required.
- Recursion is supported.


fact (int n)
{ if (n≤1) return 1;
else
return (n * fact (n-1));
}
fact (6)
Dynamic Allocation
return 1
n x f(n-1)
n x f(n-1)
n x f(n-1)
n x f(n-1)
n x f(n-1)
n=1
n=2
n=3
n=4
n=5
n=6
return 720

LECTURE NO:3

ACCESSING NON LOCAL NAMES IN A BLOCK STRUCTURED LANGUAGE

Access to Nonlocal Data on the Stack

Introduction**:** It is very important to know that how procedures access their data, specially the mechanism for finding data used within a procedure p but that does not belong to p. Access becomes more complicated in languages where procedures can be declared inside other procedures.

We therefore begin with the simple case of C functions, and then introduce a language, ML, that permits both nested function declarations and functions as "first-class objects;" that is, functions can take functions as arguments and return functions as values. This capability can be supported by modifying the implementation of the run-time stack.

Data Access without Nested Procedures:

In the C family of languages, all variables are defined either within a single function or outside any function ("globally").

Most importantly, it is impossible to declare one procedure whose scope is entirely within another procedure. Rather, a global variable v has a scope consisting of all the functions that follow the declaration of v, except where there is a local definition of the identifier v.

Variables declared within a function have a scope consisting of that function only, or part of it, if the function has nested blocks. For languages that do not allow nested procedure declarations, allocation of storage for variables and access to those variables is simple:

1. Global variables are allocated static storage. The locations of these variables remain fixed and are known at compile time. So to access any variable that is not local to the currently executing procedure, we simply use the statically determined address.

2. Any other name must be local to the activation at the top of the stack. We may access these variables through the top SP pointer of the stack.

An important benefit of static allocation for global is that declared procedures may be passed as parameters or returned as results (in C, a pointer to the function is passed), with no substantial change in the data-access strategy.

Issues with Nested Procedures:

Access becomes far more complicated when a language allows procedure declarations to be nested and also uses the normal static scoping rule; that is, procedure can access variables of the procedures whose declarations surround its own declaration, following the nested scoping rule described for blocks.

The reason is that knowing at compile time that the declaration of p is immediately nested within q does not tell us the relative positions of their activation records at run time. In fact, since either p or q or both may be recursive, there may be several activation records of p and/or q on the stack.

Finding the declaration that applies to a nonlocal name x in a nested procedure is a static decision; it can be done by an extension of the static-scope rule for blocks. Suppose x is declared in the enclosing procedure q. finding the relevant activation of q from an activation of p is a dynamic decision; it requires additional run-time information about activations. One possible solution to this problem is to use "access links‖.

A Language with Nested Procedure Declarations: The C family of languages and many other familiar languages do not support nested procedures, so we introduce one that does. The history of nested procedures in languages is long.

ML is a functional language, meaning that variables, once declared and initialized, are not changed. There are only a few exceptions, such as the array, whose elements can be changed by special function calls.

Variables are defined, and have their unchangeable values initialized, by a statement of the form:

val (name) = (expression)

Functions are defined using the syntax:

fun (name) ((arguments)) = (body)

For function bodies we shall use let-statements of the form:

let (list of definitions) in (statements) end

The definitions are normally v a l or fun statements. The scope of each such definition consists of all following definitions, up to the in, and all the statements up to the end. Most importantly, function definitions can be nested. For example, the body of a function p can contain a let-statement that includes the definition of another (nested) function q. Similarly, q can have function definitions within its own body, leading to arbitrarily deep nesting of functions.

LECTURE NO:4

Heap Management

Introduction: Heap used for dynamically allocated memory, its important operations includes allocation and de-allocation. In C, Pascal, and Java allocation is done via the ―new‖ operator and C allocation is done via the ―malloc‖ function call.

De-allocation is done either automatically. The heap is the portion of the store that is used for data that lives indefinitely, or until the program explicitly deletes it.

While local variables typically become inaccessible when their procedures end, many languages enable us to create objects or other data whose existence is not tied to the procedure activation that creates them.

For example, both C and Java give the programmer new to create objects that may be passed — or pointers to them may be passed — from procedure to procedure, so they continue to exist long after the procedure that created them is gone. Such objects are stored on a heap.

Heap Memory Manager: The memory manager keeps track of all the free space in heap storage at all times. It performs two basic functions:

- **Allocation.** When a program requests memory for a variable or object, the memory manager produces a chunk of contiguous heap memory of the requested size.

  If possible, it satisfies an allocation request using free space in the heap; if no chunk of the needed size is available, it seeks to increase the heap storage space by getting consecutive bytes of virtual memory from the operating system. If space is exhausted, the memory manager passes that information back to the application program.

- **De-allocation**. The memory manager returns de-allocated space to the pool of free space, so it can reuse the space to satisfy other allocation requests.

Memory managers typically do not return memory to the operating system, even if the program's heap usage drops. Memory management would be simpler if (a) all allocation requests were for chunks of the same size, and (b) storage were released predictably, say, first-allocated first-de-allocated.

There are some languages, such as Lisp, for which condition (a) holds; pure Lisp uses only one data element — a two pointer cell — from which all data structures are built. Condition (b) also holds in some situations, the most common being data that can be allocated on the run-time stack.

However, in most languages, neither (a) nor (b) holds in general. Rather, data elements of different sizes are allocated, and there is no good way to predict the lifetimes of all allocated objects.

Thus, the memory manager must be prepared to service, in any order, allocation and de-allocation requests of any size, ranging from one byte to as large as the program's entire address space.

Here are the properties we desire of memory managers:

- **Space Efficiency.** A memory manager should minimize the total heap space needed by a program. Doing so allows larger programs to run in a fixed virtual address space. Space efficiency is achieved by minimizing "fragmentation,".
- **Program Efficiency.** A memory manager should make good use of the memory subsystem to allow programs to run faster. As the time taken to execute an instruction can vary widely depending on where objects are placed in memory. Fortunately, programs tend to exhibit "locality," which refers to the non-random clustered way in which typical programs access memory. By attention to the placement of objects in memory, the memory manager can make better use of space and, hopefully, make the program run faster.
- **Low Overhead**. Because memory allocations and de-allocations are frequent operations in many programs, it is important that these operations be as efficient as possible. That is, we wish to minimize the overhead — the fraction of execution time spent performing allocation and de-allocation. Notice that the cost of allocations is dominated by small requests; the overhead of managing large objects is less important, because it usually can be amortized over a larger amount of computation.

LECTURE NO:5

PARAMETER PASSING

What is parameter passing

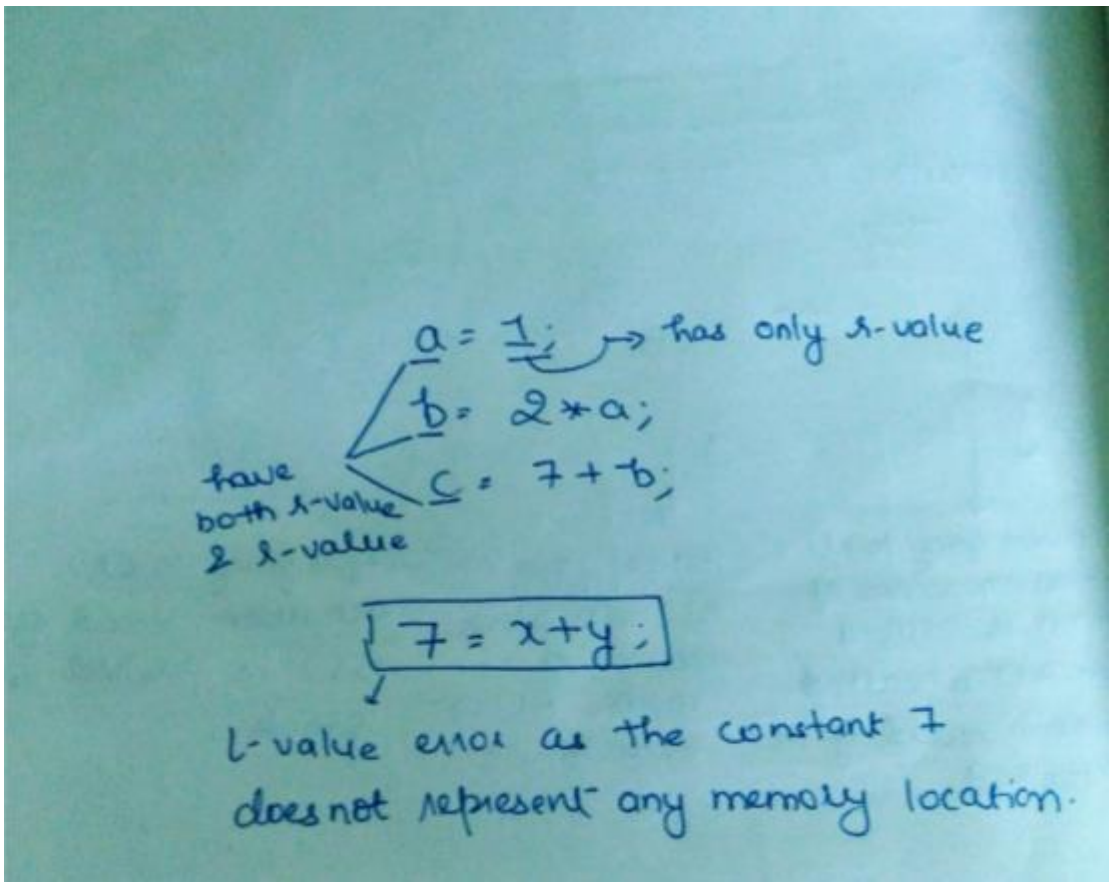
The communication medium among procedures is known as parameter passing. The values of the variables from a calling procedure are transferred to the called procedure by some mechanism.

Accessing local names in a block structured languages

Basic terminology:

- **R- value:** The value of an expression is called its R-value. The value contained in a single variable also becomes an R-value if its appear on the right side of the assignment operator. R-value can always be assigned to some other variable.

- **L-value:** The location of the memory(address) where the expression is stored is known as the l-value of that expression. It always appears on the left side if the assignment operator.



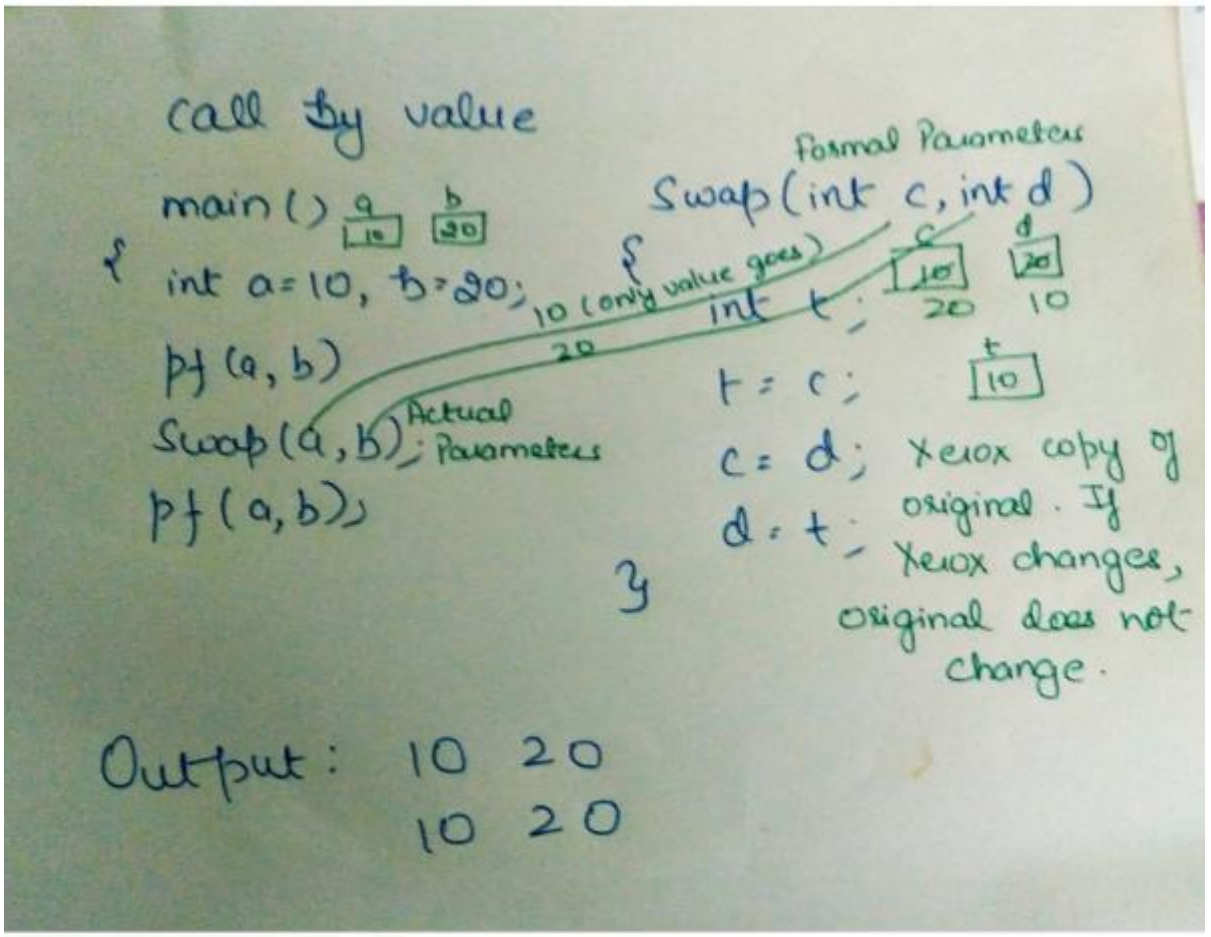
I. Formal Parameter: Variables that take the information passed by the caller procedure are called formal parameters. These variables are declared in the definition of the called function.

II. Actual Parameter: Variables whose values and functions are passed to the called function are called actual parameters. These variables are specified in the function call as arguments.

Different ways of passing the parameters to the procedure

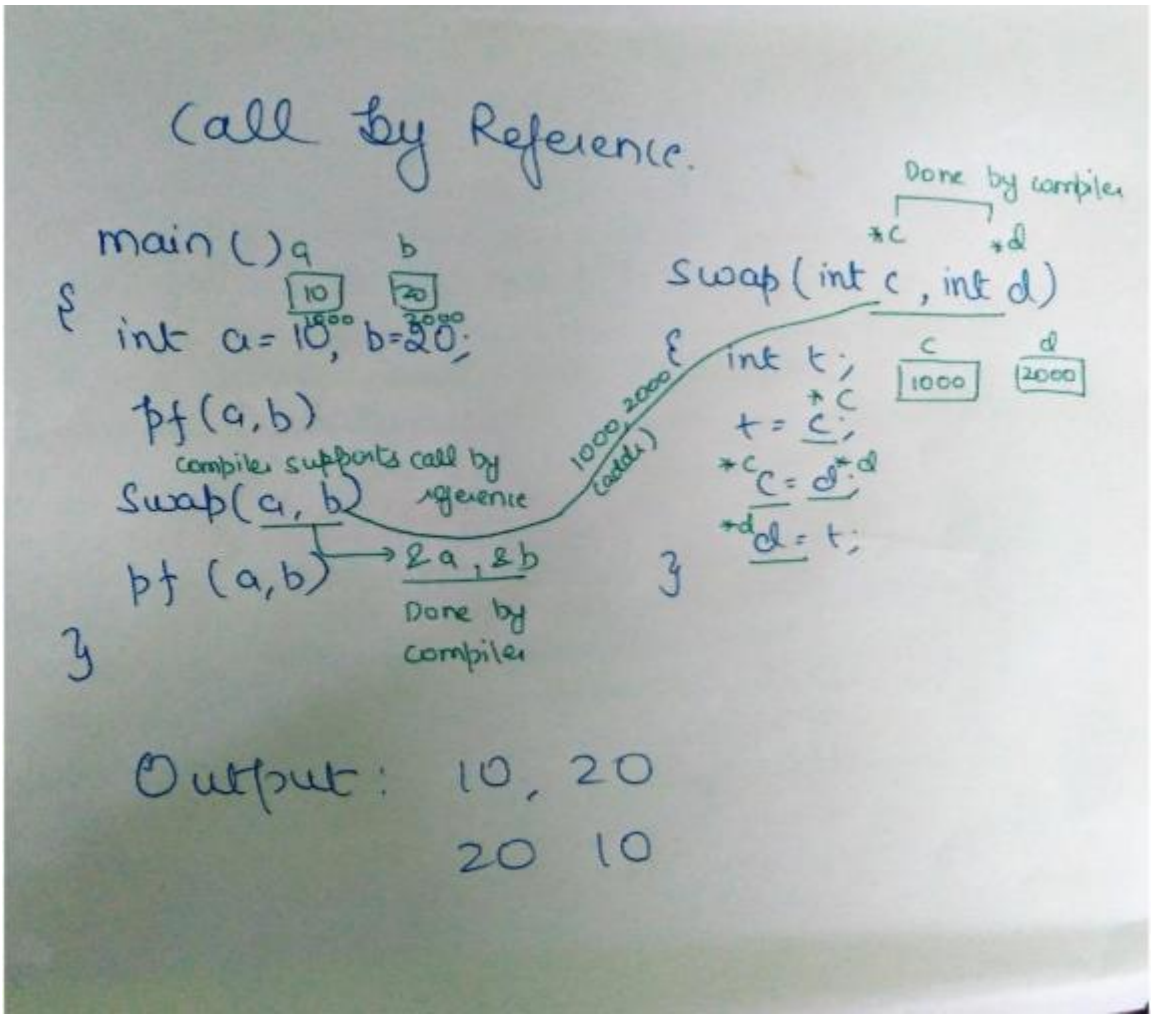
- **Call by Value**

  In call by value the calling procedure pass the R-value of the actual parameters and the compiler puts that into called procedure's activation record. Formal parameters hold the values passed by the calling procedure, thus any changes made in the formal parameters does not affect the actual parameters.



Call by Reference: In call by reference the formal and actual parameters refers to same memory location. The l-value of actual parameters is copied to the activation record of the called function. Thus the called function has the address of the actual parameters.

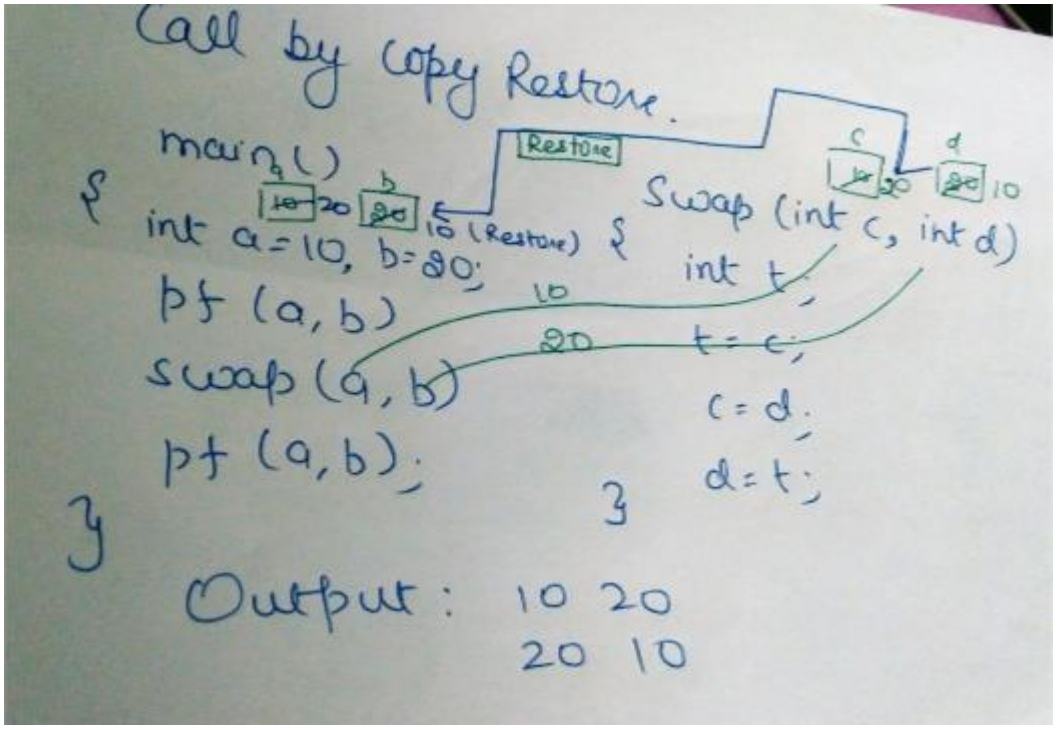
If the actual parameters do not have a l-value (e.g.- i+3) then it is evaluated in a new temporary location and the address of the location is passed. Any changes made in the formal parameter is reflected in the actual parameters (because changes are made at the address).



Call by Copy Restore

In call by copy restore compiler copies the value in formal parameters when the procedure is called and copy them back in actual parameters when control returns to the called function. The R-values are passed and on return R-value of formals are copied into

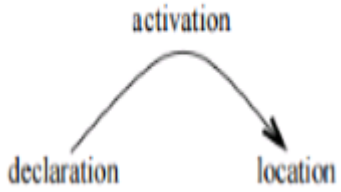
l-value of actuals.



- **Call by Name** In call by name the actual parameters are substituted for formals in all the places formals occur in the procedure. It is also referred as lazy evaluation because evaluation is done on parameters only when needed.

LECTURE NO:6

Activation Records



- Recall that an activation is an execution of a subprogram. In most languages, local variables are allocated when this execution begins (activation time).
- The storage (for formals, local variables, function results etc.) needed for an activation is organized as an activation record (or frame).
- In a language with recursion, each simultaneous activation of a recursive subprogram can have different parameters, different values for local variables, return a different result.
    - Each activation needs its own activation record
    - The number of activation records needed isn‘t known at compile time
    - Allocation of activation records is dynamic, and local variables are stack dynamic (unless declared static).
- A program is a sequence of instructions combined into a number of procedures.
- Instructions in a procedure are executed sequentially.
- A procedure has a start and an end delimiter and everything inside it is called the body of the procedure.
- The procedure identifier and the sequence of finite instructions inside it make up the body of the procedure.
- The execution of a procedure is called its activation.
- An activation record contains all the necessary information required to call a procedure.
- An activation record may contain the following units (depending upon the source language used).

Activate Record

- It is used to store the current record and the record is been stored in the stack.
- It contains return value .After the execution the value is been return.
- It can be called as return value. Parameter
- It specifies the number of parameters used in functions.

Temporaries	**Stores temporary and intermediate values of an expression.**
Local Data	Stores local data of the called procedure.
Machine Status	Stores machine status such as Registers, Program Counter etc., before the procedure is called.
Control Link	Stores the address of activation record of the caller procedure.
Access Link	Stores the information of data which is outside the local scope.
Actual Parameters	Stores actual parameters, i.e., parameters which are used to send input to the called procedure.
Return Value	Stores return values.

- Whenever a procedure is executed, its activation record is stored on the stack, also known as control stack.
- When a procedure calls another procedure, the execution of the caller is suspended until the called procedure finishes execution.
- At this time, the activation record of the called procedure is stored on the stack.
- We assume that the program control flows in a sequential manner and when a procedure is called, its control is transferred to the called procedure.

- When a called procedure is executed, it returns the control back to the caller. This type of control flow makes it easier to represent a series of activations in the form of a tree, known as the activation tree.

LECTURE NO: 7

SYMBOL TABLE ORGANISATION

Symbol Table in Compiler

Symbol table is an important data structure used in a compiler. Symbol table is used **to store the information about the occurrence of various entities** such as objects, classes, variable name, interface, function name etc. it is used by both the analysis and synthesis phases.

Symbol Table is an important data structure created and maintained by the compiler in order to keep track of semantics of variable i.e. it stores information about scope and binding information about names, information about instances of various entities such as variable and function names, classes, objects, etc.

- It is built in lexical and syntax analysis phases.
- The information is collected by the analysis phases of compiler and is used by synthesis phases of compiler to generate code.
- It is used by compiler to achieve compile time efficiency.
- It is used by various phases of compiler as follows: -

1. **Lexical Analysis:** Creates new table entries in the table, example like entries about token.
2. **Syntax Analysis:** Adds information regarding attribute type, scope, dimension, line of reference, use, etc in the table.
3. **Semantic Analysis:** Uses available information in the table to check for semantics i.e. to verify that expressions and assignments are semantically correct (type checking) and update it accordingly.
4. **Intermediate Code generation:** Refers symbol table for knowing how much and what type of run-time is allocated and table helps in adding temporary variable information.
5. **Code Optimization:** Uses information present in symbol table for machine dependent optimization.
6. **Target Code generation:** Generates code by using address information of identifier present in the table.

Symbol Table entries – Each entry in symbol table is associated with attributes that support compiler in different phases.

Items stored in Symbol table:

- Variable names and constants
- Procedure and function names
- Literal constants and strings
- Compiler generated temporaries
- Labels in source languages

Information used by compiler from Symbol table:

- Data type and name
- Declaring procedures
- Offset in storage
- If structure or record then, pointer to structure table.
- For parameters, whether parameter passing by value or by reference
- Number and type of arguments passed to function
- Base Address

Operations of Symbol table – The basic operations defined on a symbol table include:

Operation	Function
allocate	to allocate a new empty symbol table
free	to remove all entries and free storage of symbol table
lookup	to search for a name and return pointer to its entry
insert	to insert a name in a symbol table and return a pointer to its entry
set_attribute	to associate an attribute with a given entry
get_attribute	to get an attribute associated with a given entry

LECTURE NO:8

DATA STRUCTURE USED IN SYMBOL TABLE

Implementation of Symbol table –

Following are commonly used data structure for implementing symbol table: -

1. **List –**

- In this method, an array is used to store names and associated information.
- A pointer **"available"** is maintained at end of all stored records and new names are added in the order as they arrive
- To search for a name, we start from beginning of list till available pointer and if not found we get an error **"use of undeclared name".**
- While inserting a new name we must ensure that it is not already present otherwise error occurs i.e. **"Multiple defined name".**
- Insertion is fast O (1), but lookup is slow for large tables – O(n) on average
- Advantage is that it takes minimum amount of space.

2. Linked List –

- This implementation is using linked list. A link field is added to each record.
- Searching of names is done in order pointed by link of link field.
- A pointer **"First"** is maintained to point to first record of symbol table.
- Insertion is fast O (1), but lookup is slow for large tables – O(n) on average

3. Hash Table –

- In hashing scheme two tables are maintained – a hash table and symbol table and is the most commonly used method to implement symbol tables.
- A hash table is an array with index range: 0 to table size – 1.
- These entries are pointer pointing to names of symbol table.

- To search for a name, we use hash function that will result in any integer between 0 to table size – 1.
- Insertion and lookup can be made very fast – O (1).
- Advantage is quick search is possible and disadvantage is that hashing is complicated to implement.

4. Binary Search Tree –

- Another approach to implement symbol table is to use binary search tree i.e. we add two link fields i.e. left and right child.
- All names are created as child of root node that always follow the property of binary search tree.
- Insertion and lookup are O ($\log_2$ n) on average.